chapter 03: Compute architecture and scheduling

A **GPU** is organized into **Streaming Multiprocessors (SMs)**, which consist of multiple processing blocks sharing control logic and memory resources. When a grid launches, blocks are assigned to SMs arbitrarily, enabling **transparent scalability** but with the limitation that threads in different blocks cannot synchronize. Threads are assigned to SMs block-by-block and partitioned into **warps** that execute following the **SIMD model**, with divergent threads executed in passes. An SM can have many more threads assigned than it can execute simultaneously, allowing other warps to wait for long-latency operations without impacting throughput. This ratio is called **occupancy** - higher occupancy means better latency hiding. Each **CUDA device** imposes different resource limitations per SM (blocks, threads, registers, etc.) that can limit occupancy, though CUDA C allows querying available GPU resources at runtime.

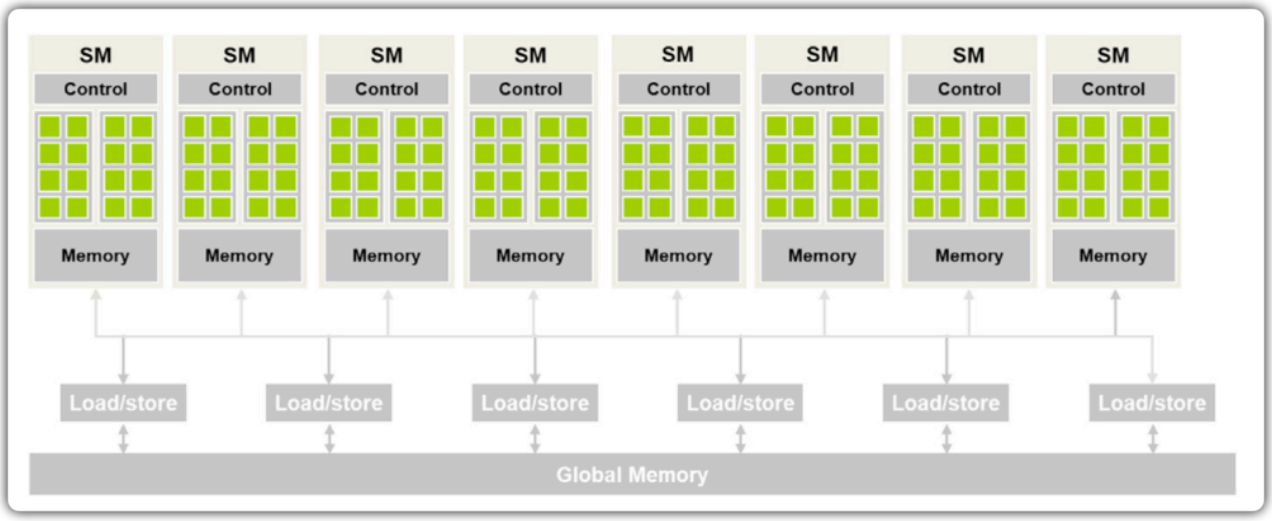


Terminology	Explanation
GPU Architecture	The design of the hardware components of a GPU. It includes the organization of the computational cores, memory system, and interconnections between them.
Streaming Multiprocessor (SM)	A group of computational cores on a GPU. Threads are scheduled to execute on these cores. Each SM has its own shared memory and L1 cache.

Terminology	Explanation
Block Scheduling	**The process of assigning thread blocks to SMs for execution.** All threads in a block are guaranteed to execute on the same SM.
Synchronization	**A mechanism that ensures that all threads in a block or grid have reached a certain point in the execution before any thread can proceed.** This is necessary to avoid data races.
Transparent Scalability	**The ability of a program to run correctly and efficiently on different GPUs with different numbers of SMs.** This is achieved by using the same grid and block dimensions regardless of the underlying hardware.
Warp	**A group of 32 threads that are scheduled to execute together on an SM.** All threads in a warp execute the same instruction at the same time.
SIMD Hardware	**Hardware that supports the simultaneous execution of the same instruction on multiple data elements.** GPUs are examples of SIMD hardware. The warp size is determined by the SIMD width of the hardware.
Control Divergence	**A phenomenon that occurs when threads in the same warp take different execution paths.** This can happen due to conditional statements or loops. Control divergence can reduce the performance of a program because threads in the same warp that are not taking the same execution path cannot execute in parallel.
Warp Scheduling	**The process of selecting which warps to execute on an SM.** Warps that are ready to execute are placed in a queue, and the warp scheduler selects the next warp to execute from the queue. To achieve latency tolerance, GPUs often use a technique called **zero-cycle context switching** to quickly switch between warps that are waiting for long-latency operations.
Latency Tolerance	**The ability of a GPU to hide the latency of long-latency operations by switching to other warps that are ready to execute.** This is achieved by having many warps available for execution.
Resource Partitioning	**The process of dividing the resources of an SM among the thread blocks that are executing on it.** These resources include shared memory, registers, and computational cores.
Occupancy	**The ratio of the number of active warps on an SM to the maximum number of warps that can be active on the SM.** Occupancy is a measure of how well the resources of an SM are being utilized. To achieve high performance, it is important to maximize the occupancy of the GPU by carefully tuning the resource usage of the kernels.

Architecture of a modern GPU





- **CUDA-capable GPU Architecture Overview**
  - *Organization*
    - Array of highly threaded streaming multiprocessors (SMs)

1. Does Each Thread Run on One CUDA Core?

- **Yes, but not always simultaneously.**
  - Each thread is mapped to a CUDA core for execution, but **not all threads run simultaneously.**
  - CUDA cores are the arithmetic units inside a **Streaming Multiprocessor (SM)** that execute thread instructions.
  - An SM can have **fewer CUDA cores than the total threads in a block**, meaning threads take turns executing based on the hardware's **warp scheduler**.

---

2. What Happens When the Number of Threads in a Block Is Bigger Than the Number of CUDA Cores in an SM?

If the number of threads in a block exceeds the number of CUDA cores in an SM, **time-slicing and scheduling** come into play:

Key Details:

1. **Thread Scheduling and Warps:**
   - Threads in a block are divided into groups of **32 threads called a warp**.
   - Each warp executes instructions in lockstep (all 32 threads perform the same instruction simultaneously).
   - If a block has more threads than the SM's CUDA cores can handle in a single cycle, warps are **executed in multiple cycles.**
2. **Time Multiplexing:**
   - SMs use a **warp scheduler** to decide which warp to run at a given time.
   - Warps are executed in a round-robin or availability-based manner.
   - While some warps are waiting (e.g., for memory operations), others are executed to maximize resource utilization.
3. **CUDA Cores per SM:**
   - For example, consider an NVIDIA GPU:
     - An SM might have **128 CUDA cores**.
     - A block might contain **256 threads**.
   - Here, only **4 warps (128 threads)** can run simultaneously, and the other warps will wait for their turn.
4. **Latency Hiding:**
   - While some warps are stalled (e.g., waiting for memory operations), the GPU scheduler switches to ready warps, hiding memory latency and maximizing throughput.
   - The large number of threads ensures that the SM remains busy, even if some threads are waiting.

Example Scenario

- Suppose an SM has **128 CUDA cores**.
- A block has **256 threads**.
- Threads are divided into **8 warps** (256 ÷ 32 = 8).

Execution:

- At any given moment, the SM executes **4 warps simultaneously** (128 CUDA cores can handle 4 × 32 threads).
- The remaining 4 warps wait in the queue and are executed in subsequent cycles, as threads from the active warps complete their operations or are stalled.

Summary of How Threads Are Handled:

Situation	What Happens
Threads ≤ CUDA cores in an SM	All threads can execute simultaneously.
Threads > CUDA cores in an SM	Warps are scheduled in cycles, ensuring all threads eventually execute.
Warp Stalls (e.g., memory latency)	Other warps execute while waiting warps are stalled, hiding latency.

        - Each SM contains:
            - Multiple streaming processors/CUDA cores
            - Shared control logic
            - Shared memory resources
    - *Example Specifications*
        - Ampere A100 GPU:
            - 108 SMs
            - 64 cores per SM
            - Total: **6912 cores**
- **Memory Architecture**
    - *On-chip Memory*
        - Different memory structures labeled as "Memory"
    - *Off-chip Memory (Global Memory)*
        - Capacity: Multiple gigabytes
        - Memory Types:
            - Older GPUs: Graphics double data rate synchronous DRAM
            - Newer GPUs (NVIDIA Pascal+): HBM/HBM2
                - DRAM modules integrated with GPU package

Block scheduling

- **CUDA Kernel Execution Process**
  - When a kernel is called:
    - CUDA runtime system launches a grid of threads
    - These threads execute the kernel code
    - Threads are assigned to SMs block-by-block
    - All threads in a block go to the same SM
  - example

Scenario: Vector Addition on CUDA

Suppose we want to add two vectors, A and B, element-wise to produce a result vector C. Each vector has 1,024 elements. We'll perform this addition using CUDA.

Step-by-Step Breakdown:

Kernel Code

Here's a simple CUDA kernel for vector addition:

```cpp
__global__ void vectorAdd(float* A, float* B, float* C, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

Kernel Launch

We launch the kernel with the following configuration:

```cpp
int N = 1024;                // Size of vectors
int threadsPerBlock = 256;   // Number of threads per block
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock; // Total blocks

vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(A, B, C, N);
```

- **Threads per block:** 256
- **Total threads:** $4 \times 256 = 1,024$
- **Blocks per grid:** 4 (to cover all 1,024 elements)

Key Concepts:

1. Launching a Grid of Threads

When `vectorAdd` is called, the **CUDA runtime system launches a grid** consisting of:

- **4 blocks**, each with **256 threads**.
- These threads collectively perform the addition operation on the elements of A and B.

2. Kernel Execution

Each thread calculates its **global index** using:

$$\text{idx} = \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$$

- Threads work independently to compute $C[idx] = A[idx] + B[idx]$, with each thread handling a single element.

For example:

- **Block 0, Thread 0** computes $C[0] = A[0] + B[0]$.
- **Block 0, Thread 255** computes $C[255] = A[255] + B[255]$.
- **Block 1, Thread 0** computes $C[256] = A[256] + B[256]$.

3. Assignment of Blocks to SMs

When the kernel is launched, **blocks are assigned to Streaming Multiprocessors (SMs)**:

- Suppose the GPU has **2 SMs**.
- The **4 blocks** are distributed across the SMs:
  - Block 0 and Block 1 → Assigned to SM 0.
  - Block 2 and Block 3 → Assigned to SM 1.

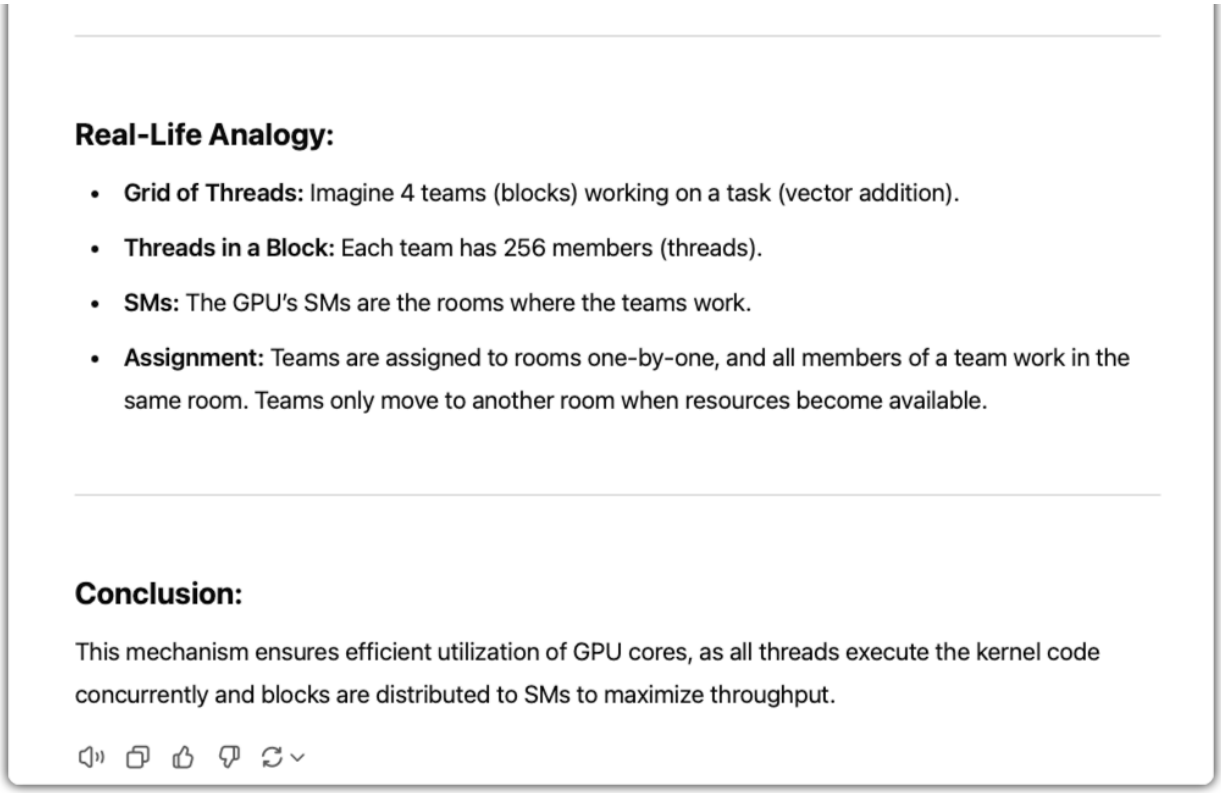
4. Threads in a Block Share the Same SM

All threads within a block are executed on the same SM:

- **Block 0's threads (Thread 0 to Thread 255)** are executed on SM 0.
- **Block 1's threads (Thread 0 to Thread 255)** are also executed on SM 0, but only after Block 0 finishes or when resources are available.
- Similarly, Block 2 and Block 3's threads are executed on SM 1.

Key Observations:

1. **Thread Independence:**
   Each thread independently handles one element of the input vectors, ensuring parallel computation.
2. **Block Assignment to SMs:**
   CUDA assigns blocks to SMs **block-by-block**, ensuring that the work is distributed among SMs based on available resources.
3. **Threads Within a Block:**
   All threads in a block execute on the **same SM**, allowing them to share resources like **shared memory** and **registers**.
4. **Parallelism at Two Levels:**
   - **Block-Level Parallelism:** Multiple blocks execute on different SMs simultaneously.
   - **Thread-Level Parallelism:** Threads within a block execute concurrently using the SM's cores.



Real-Life Analogy:

- **Grid of Threads:** Imagine 4 teams (blocks) working on a task (vector addition).
- **Threads in a Block:** Each team has 256 members (threads).
- **SMs:** The GPU's SMs are the rooms where the teams work.
- **Assignment:** Teams are assigned to rooms one-by-one, and all members of a team work in the same room. Teams only move to another room when resources become available.

Conclusion:

This mechanism ensures efficient utilization of GPU cores, as all threads execute the kernel code concurrently and blocks are distributed to SMs to maximize throughput.

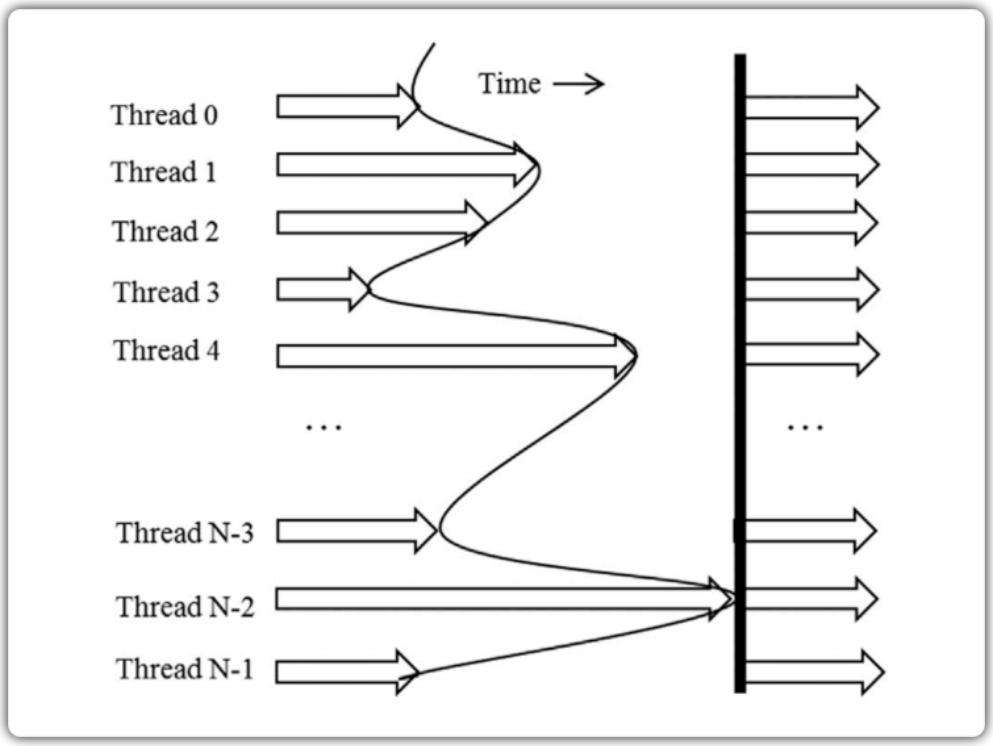
- **Block Assignment to SMs**
  - **Multiple blocks can be assigned to one SM**
    - Example: Three blocks per SM
  - *Limited by hardware resources*
    - Only certain number of blocks per SM
- **Runtime Block Management**
  - *Limited total concurrent blocks*
    - Due to SM count and blocks-per-SM limits
  - Most grids have more blocks than can run at once
  - Runtime system:
    - Maintains list of pending blocks
    - Assigns new blocks as others complete
- **Thread Block Interactions**
  - *Key benefit*: Same-block threads run together on one SM
  - **Enables special interactions within blocks**:
    - Barrier synchronization
    - Access to shared low-latency memory
  - These interactions not possible across different blocks

Synchronization and transparent scalability

- **CUDA Thread Synchronization**

- CUDA provides __syncthreads() function for barrier synchronization
  - When called, thread waits at that location
  - Ensures all threads in block complete before proceeding

- **Execution Process**
  - Block contains N threads



  - Time flows left to right
  - Thread arrival pattern:
    - Some threads reach barrier early
    - Some threads reach barrier late
    - Early threads wait for late arrivals
    - All proceed together once last thread arrives
  - Key principle: "no one is left behind"

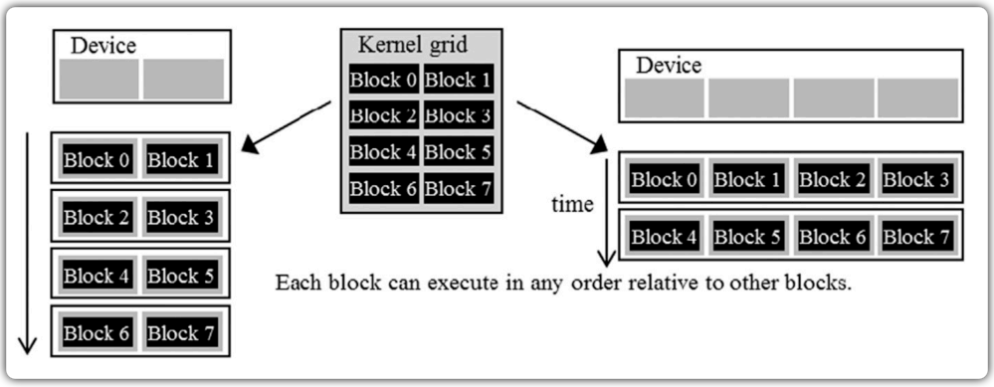
- **CUDA Barrier Synchronization (__syncthreads())**
  - **Key Rule**: All threads in a block must execute __syncthreads() if present

```
void incorrect_barrier_example(int n) {
   // Some computation for even threads
   if (threadIdx.x % 2 == 0) {
      // Do work for even threads
      __syncthreads();
   }
   else {
      // Do work for odd threads
      __syncthreads();
   }
}
```

- Usage in Conditional Statements
  - **If Statement**: Either *all* threads execute __syncthreads() or *none* do
  - **If-Then-Else**: All threads must take *same path* containing __syncthreads()
  - **Warning**: Different barriers in different paths lead to undefined behavior
- Example Issue
  - Code with __syncthreads() in both if-else paths
  - Even threadIdx.x values take one path, odd take another
  - Creates two different barriers
  - Violates usage rules
  - Can cause:
    - Incorrect results
    - Deadlocks (threads waiting forever)
- Execution Constraints
  - Threads in block must execute in close time proximity
  - All threads need guaranteed resource access

    > Not only do all threads in a block have to be assigned to the same SM, but also they need to be assigned to that SM simultaneously. That is, a block can begin execution only when the runtime system has secured all the resources needed by all threads in the block to complete execution. This ensures the time proximity of all threads in a block and prevents an excessive or even indefinite waiting time during barrier synchronization.

  - CUDA Runtime System requirements:
    - All block threads assigned to same SM
    - Simultaneous assignment
    - Resources secured before execution begins
- Design Tradeoffs
  - No barrier synchronization between different blocks
  - Benefits:
    - Blocks execute in any order
    - Enables scalable implementations
- Implementation Flexibility



- Supports various execution scenarios:
  - Low-cost systems: Few blocks at once
  - High-end systems: Hundreds of blocks simultaneously

- Transparent Scalability

  - Same code runs on different hardware configurations
  - Advantages:
    - Adaptable to different market segments
    - Mobile: Lower power, slower execution
    - Desktop: Higher power, faster execution
  - Benefits:
    - Reduced developer burden
    - Improved application usability

Warps and SIMD hardware

- **Block Execution and Thread Behavior**

  - Blocks can execute in any order relative to each other

    - Enables transparent scalability across different devices

Transparent Scalability in GPU Programming

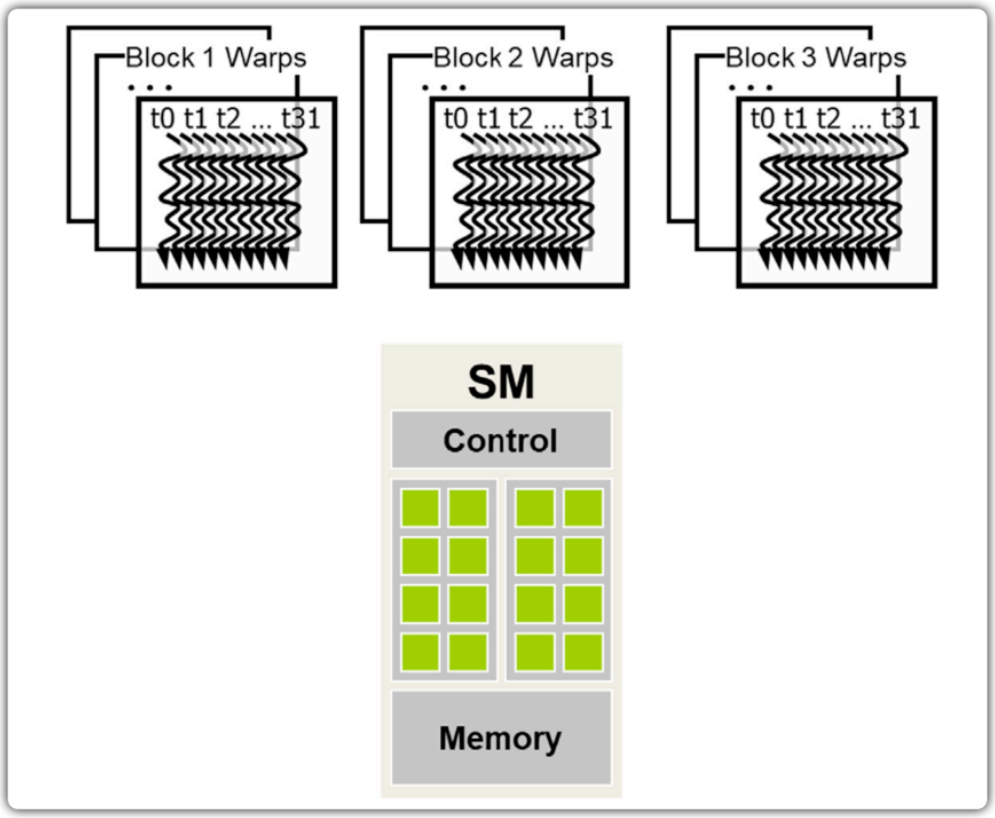
Transparent scalability refers to the ability of a program or algorithm to scale across varying hardware resources (e.g., different numbers of Streaming Multiprocessors (SMs) or CUDA cores) without requiring changes to the code. In the GPU programming domain, this concept is crucial because GPUs come in various configurations, with differing numbers of SMs, CUDA cores, and memory sizes.

Key Points of Transparent Scalability:

1. **Grid and Block Abstraction**:
   - CUDA programming uses **grids** and **blocks** to define parallel workloads.
   - The programmer specifies the total number of threads, organized into thread blocks, without needing to know the exact hardware configuration.
   - The CUDA runtime system distributes these threads to the available hardware (e.g., SMs and CUDA cores).
2. **Dynamic Resource Allocation**:
   - The hardware and CUDA runtime manage thread block assignment dynamically. For example:
     - On a GPU with 4 SMs, thread blocks are distributed among the 4 SMs.
     - On a GPU with 8 SMs, the same code runs, but now thread blocks are distributed among 8 SMs, improving throughput.
3. **Independent Execution of Thread Blocks**:
   - Thread blocks are designed to execute independently. This independence allows the program to adapt to different hardware configurations:
     - If more SMs are available, more thread blocks can run in parallel.
     - If fewer SMs are available, thread blocks run sequentially but still complete correctly.
4. **Scalability Without Rewriting Code**:
   - Programs written with CUDA's grid and block structure scale transparently to GPUs with different hardware capabilities.
   - For example:
     - A kernel launched with 1,024 thread blocks will utilize 4 SMs on one GPU and 10 SMs on another GPU without modification.
5. **Latency Hiding Through Overprovisioning**:
   - Transparent scalability ensures that there are always more thread blocks than SMs, so while some threads are waiting (e.g., on memory), other threads can execute.
   - This overprovisioning keeps the GPU fully utilized, regardless of the number of physical cores.

- Threads within a block can execute in any order
  - *Barrier synchronization* needed between phases

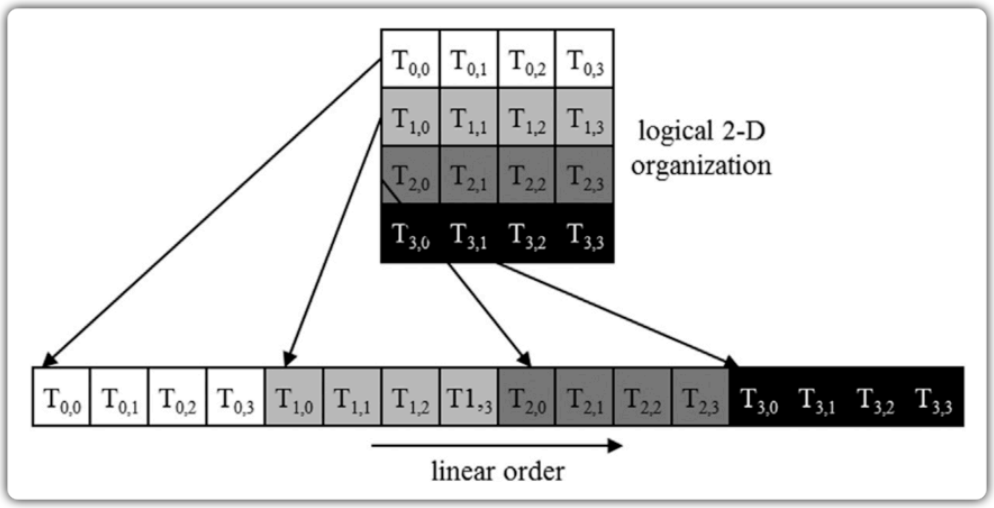
        - Kernel execution correctness should not assume thread synchronization without barriers
- **Thread Scheduling in CUDA GPUs**
    - Hardware implementation-specific concept
    - Block organization:
        - Blocks assigned to SM (Streaming Multiprocessor)
        - Further divided into *warps* (32-thread units)
        - Warp size may vary in future GPU generations
    - *Understanding warps helps optimize CUDA application performance*
- **Warp Structure and Organization**
    - Warp = unit of thread scheduling in SMs
    - Example implementation:



        - Three blocks (Block 1, 2, 3) assigned to one SM
        - Each block divided into warps
        - Warps contain 32 consecutive threadIdx values:
            - First warp: threads 0-31
            - Second warp: threads 32-63
            - And so on
    - Warp calculation example:
        - Block size: 256 threads
        - Warps per block: 256/32 = 8
        - Total warps in SM with 3 blocks: 8 * 3 = 24
- **Thread Block Partitioning into Warps**

- Based on thread indices
- For 1D blocks (using only threadIdx.x):
  - Threads within warp are consecutive and increasing
  - For warp size 32:
    - Warp 0: threads 0-31
    - Warp 1: threads 32-63
    - Warp n: threads 32×n to 32×(n+1)-1
  - **Last warp padded with inactive threads if block size not multiple of 32**
    - *Example*: 48-thread block = 2 warps, second warp has 16 inactive threads

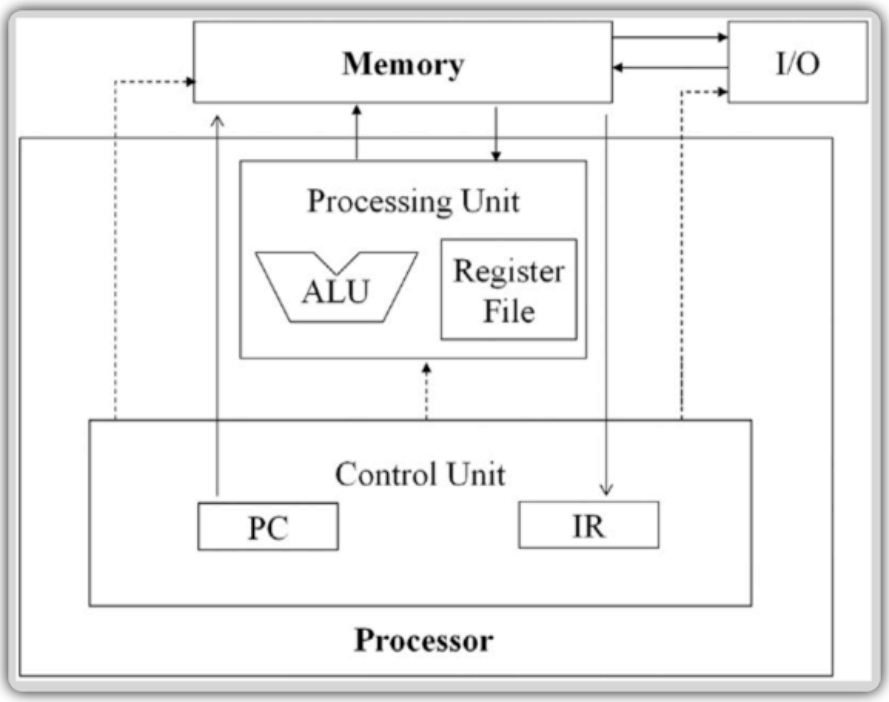
- **Multi-dimensional Block Partitioning**
  - Uses row-major linearized layout
  - Higher y/z coordinates placed after lower ones



  - For 2D blocks:
    - Threads ordered by increasing threadIdx.y
    - Within same threadIdx.y, ordered by increasing threadIdx.x
  - For 3D blocks:
    - First place all threadIdx.z = 0 threads
    - Then threadIdx.z = 1 threads, and so on
    - *Example*: 2×8×4 block = 64 threads = 2 warps
      - First warp: $T_{0,0,0}$ through $T_{0,7,3}$
      - Second warp: $T1_{0,0,0}$ through $T1_{0,7,3}$

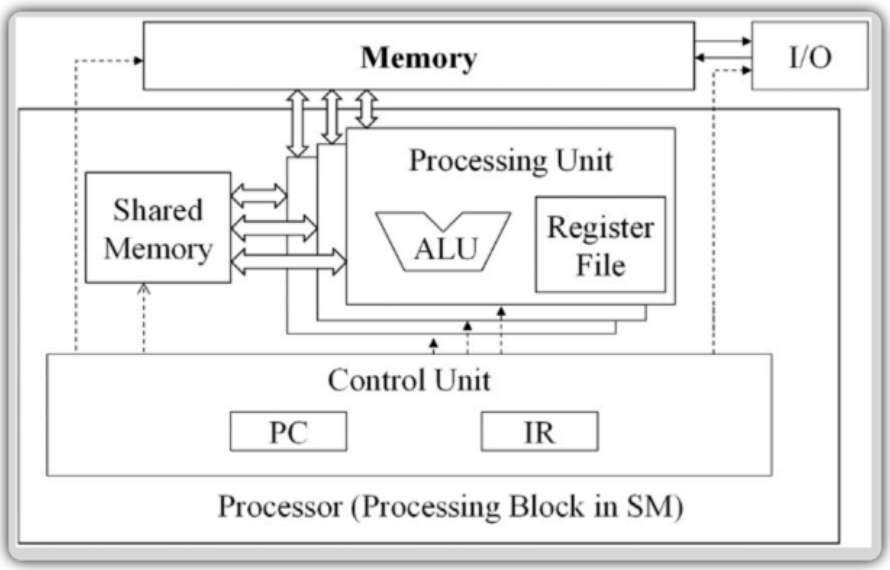
- **SM Execution Model**
  - Based on SIMD (Single-Instruction, Multiple-Data)
  - Key characteristics:
    - **One instruction fetched/executed for all threads in warp**
    - Cores grouped into processing blocks
    - **Each processing block shares instruction fetch/dispatch unit**
  - *Real Example - Ampere A100 SM*:

        - 64 cores total
        - 4 processing blocks with 16 cores each
        - Threads in same warp assigned to same processing block
        - All threads execute same instruction on different data portions
- **SIMD Advantages**
    - Control hardware cost (instruction fetch/dispatch unit) is shared across multiple execution units
    - Enables:
        - Less hardware dedicated to control
        - More hardware for increasing arithmetic throughput
- **Warp Partitioning**
    - Expected to remain popular in future implementations
    - Key characteristics:
        - Warp size can vary between implementations
        - Current CUDA devices use 32 threads per warp consistently
- **Warps and SIMD Hardware**
    - Von Neumann Model
        - In *1945*, *John von Neumann* described a model for building electronic computers



        - Based on design of pioneering *EDVAC computer*
        - This "*von Neumann Model*" became foundation for virtually all modern computers
        - Key Components

        - *I/O (input/output)*
            - Allows programs and data input/output
        - *Memory*
            - Stores program and data
        - *Control Unit*
            - Maintains *Program Counter (PC)*
                - Contains memory address of next instruction
            - Uses *Instruction Register (IR)*
                - Holds current instruction for execution
        - Known as "*stored program*" model
            - Users can modify computer behavior by storing different programs
- GPU Design and Warps
    - Modified von Neumann model adapted for GPU design

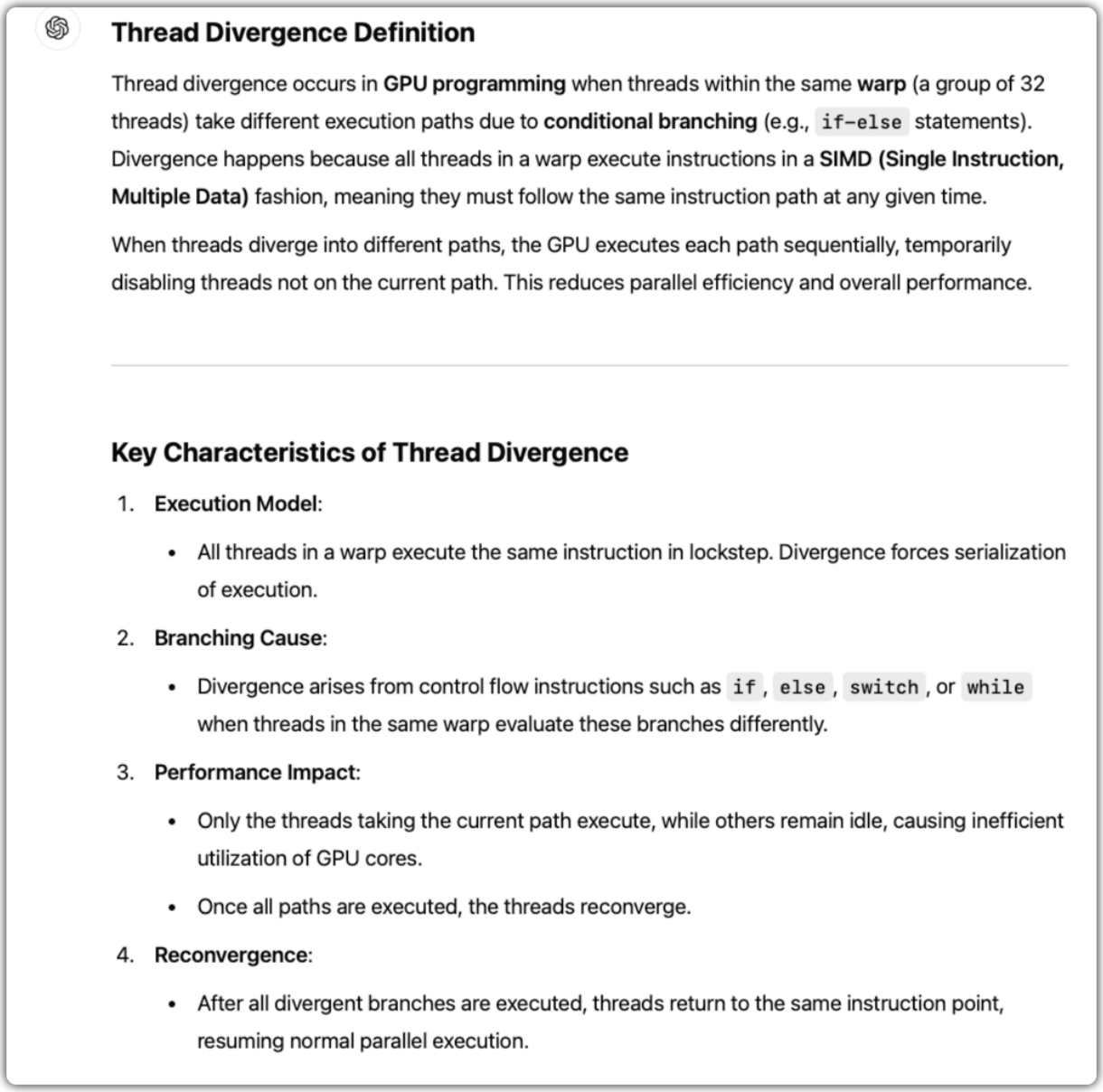


    - Processing block features:
        - *Single Control Unit*
            - Handles instruction fetching and dispatching
        - *Multiple Processing Units*
            - Each corresponds to a core in SM
            - Each executes one thread in a warp
            - All receive same control signals
    - SIMD (Single-Instruction-Multiple-Data)
        - All processing units controlled by same instruction in IR
        - Execution differences come from:
            - Different data operand values in register files
        - Example:
            - All cores execute same instruction (e.g., `add r1, r2, r3`)
            - Contents of r2 and r3 differ across processing units
    - Hardware Benefits
        - Modern control units are complex
            - Include sophisticated instruction fetching logic

        - Have instruction cache access ports
    - Sharing control unit across multiple processing units:
        - *Reduces hardware manufacturing cost*
        - *Reduces power consumption*

Control divergence

- **Optimal SIMD Execution**:
    - Best when all threads in a warp follow same execution path
    - Example: All threads taking either if-path or else-path together
- **Control Divergence**:
    - **Occurs when threads within warp take different paths**



Thread Divergence Definition

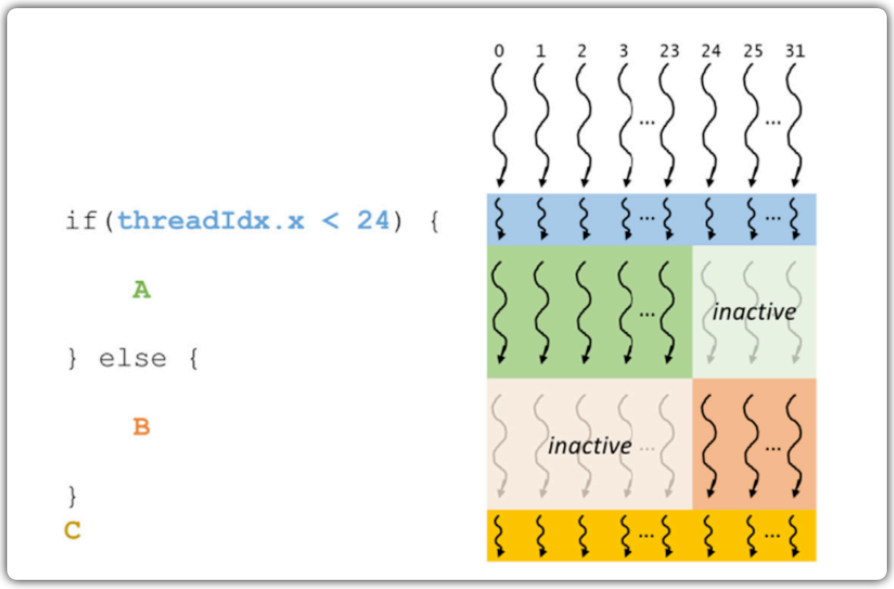
Thread divergence occurs in **GPU programming** when threads within the same **warp** (a group of 32 threads) take different execution paths due to **conditional branching** (e.g., `if-else` statements). Divergence happens because all threads in a warp execute instructions in a **SIMD (Single Instruction, Multiple Data)** fashion, meaning they must follow the same instruction path at any given time.

When threads diverge into different paths, the GPU executes each path sequentially, temporarily disabling threads not on the current path. This reduces parallel efficiency and overall performance.

Key Characteristics of Thread Divergence

1. **Execution Model**:
    - All threads in a warp execute the same instruction in lockstep. Divergence forces serialization of execution.
2. **Branching Cause**:
    - Divergence arises from control flow instructions such as `if`, `else`, `switch`, or `while` when threads in the same warp evaluate these branches differently.
3. **Performance Impact**:
    - Only the threads taking the current path execute, while others remain idle, causing inefficient utilization of GPU cores.
    - Once all paths are executed, the threads reconverge.
4. **Reconvergence**:
    - After all divergent branches are executed, threads return to the same instruction point, resuming normal parallel execution.

    - Hardware takes multiple passes, one for each path
    - Example scenario:
        - Some threads follow if-path
        - Others follow else-path
        - Results in two passes
        - Inactive threads don't take effect during other path's execution
- **Impact of Control Divergence**:
    - Preserves thread independence
    - Maintains SIMD hardware cost benefits
    - Costs:
        - Extra hardware passes required
        - Resources consumed by inactive threads
- **Example: Divergent If-Else Execution**:



    - Warp with threads 0-31:
        - Threads 0-23: Execute path A
        - Threads 24-31: Execute path B
    - Execution process:
        - **a pass refers to an independent stage or task in a computational workflow, often represented by a kernel or operation that runs on the GPU. Passes are distinct units of work that can either be dependent on one another (executed sequentially) or independent (can run concurrently if resources allow)**.
        - First pass: Threads 0-23 active (A), 24-31 inactive
        - Second pass: Threads 24-31 active (B), 0-23 inactive
        - All threads reconverge for C
- **Architecture Evolution**:
    - Pascal and earlier:

        - Sequential pass execution
    - Volta onwards:
        - Concurrent pass execution
        - Independent thread scheduling
        - Details in Volta V100 architecture whitepaper
    - comparsion

Pascal and Earlier Architectures: Sequential Pass Execution

1. **Behavior:**
   - **Sequential pass execution** means that independent computational passes (kernels or tasks) are executed **one after another** on the GPU.
   - Even if multiple tasks are independent and could run in parallel, the GPU schedules them sequentially.
2. **Implications:**
   - **Limited Utilization:**
     - If a single task does not fully utilize all the GPU resources (e.g., SMs, CUDA cores), some resources remain idle until the next task begins.
   - **Pipeline Inefficiency:**
     - Work that could theoretically be executed concurrently is delayed due to the sequential execution model.
3. **Example:**
   - Suppose we have two independent kernels, K_1 and K_2, that could run in parallel:
     - On Pascal or earlier GPUs, K_1 runs to completion, and only then does K_2 start.
     - This limits overlap and reduces throughput.

Volta and Later Architectures: Concurrent Pass Execution

1. **Behavior:**
   - **Concurrent pass execution** allows independent computational passes (kernels or tasks) to execute simultaneously on the GPU.
   - The GPU hardware and scheduler can manage multiple streams of tasks, assigning available resources dynamically to ensure maximum utilization.
2. **Key Innovations:**
   - **Independent Thread Scheduling:**
     - Volta introduced independent scheduling of warps, allowing finer control over resource allocation and execution.
   - **Dynamic Resource Partitioning:**
     - Resources (e.g., SMs, registers, memory) can be divided among multiple tasks running concurrently.
   - **Hardware Multi-Stream Execution:**
     - Support for executing kernels from different streams concurrently.
3. **Implications:**
   - **Improved Resource Utilization:**

- The GPU can run multiple independent tasks in parallel, maximizing use of all available resources.

- **Pipeline Efficiency:**
  - Different passes of a workload can overlap, reducing overall latency.

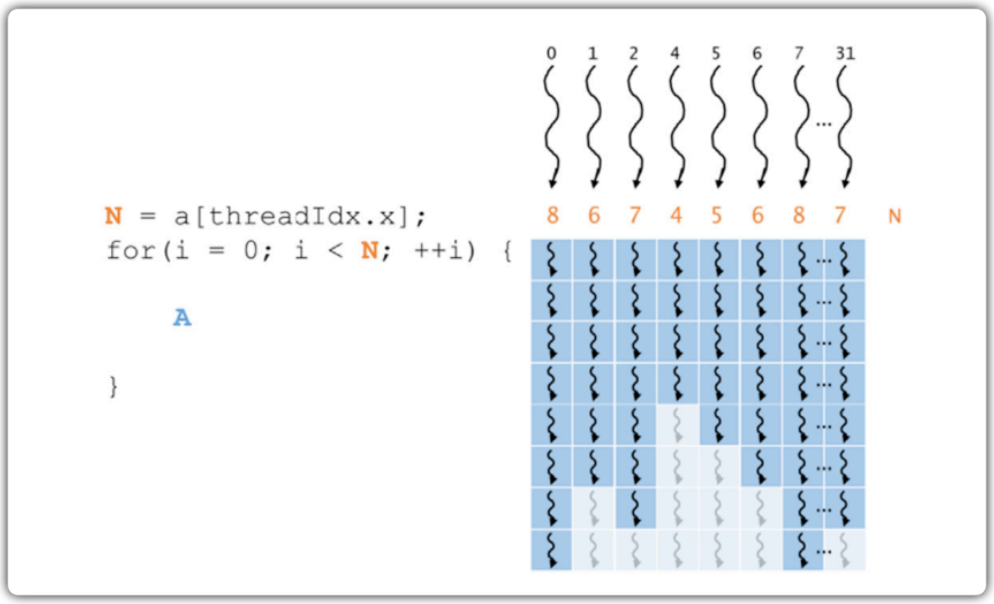
4. **Example:**
   - With the same kernels K_1 and K_2:
     - On Volta or later GPUs, K_1 and K_2 can execute simultaneously if there are sufficient resources available.
     - This results in higher throughput and shorter total execution time.

Comparison Table

Feature	Pascal and Earlier	Volta and Later
Execution Model	Sequential pass execution	Concurrent pass execution
Task Overlap	No overlap between passes	Overlap possible for independent tasks
Resource Utilization	May leave resources idle	Maximizes resource utilization
Kernel Scheduling	Coarser-grained	Finer-grained (warp-level scheduling)
Example Behavior	$K_1 \rightarrow K_2$	K_1 and K_2 run concurrently

---

- **Thread Divergence in Control Flow**



  - Example: Divergent for-loop execution in warps
    - Threads execute different numbers of loop iterations (4-8)
    - First 4 iterations: all threads active
    - Remaining iterations: some threads active, others inactive

- **Identifying Thread Divergence**
    - Check decision conditions based on `threadIdx` values
        - Example: `if(threadIdx.x . 2)` causes divergent paths
        - Loops with thread index-based conditions can cause divergence
- **Common Use Case: Boundary Conditions**
    - Handling data mapping when thread count ≠ data size
    - Example: Vector Addition
        - Vector length 1003, block size 64
        - Need 16 blocks (1024 threads) - last 21 threads disabled
        - Only last warp has control divergence
- **Performance Impact by Data Size**
    - Vector Length Impact:
        - 100 elements: 1/4 warps diverge - significant impact
        - 1000 elements: 1/32 warps diverge - ~3% impact
        - 10000+ elements: 1/313 warps diverge - <1% impact
- **2D Data Analysis**
    - Example: 62x76 Image Processing
        - 20 blocks (4x5) with 16x16 threads each
        - Total 160 warps
        - Region Analysis:
            - Region 1: 96 warps - no divergence
            - Region 2: 24 warps - all diverge
            - Region 3: Bottom warps inactive
            - Region 4: 7 warps diverge, 1 warp doesn't
        - Total: 31/160 warps have divergence
- **Important Note**
    - Cannot assume uniform execution timing in warps
    - Use `__syncwarp()` for barrier synchronization when needed

Warp scheduling and latency tolerance

- **Thread Assignment to SMs**
    - More threads assigned to SM than available cores
    - SM can execute only subset of threads at any time
    - Earlier GPUs: One instruction per warp at a time
    - Recent GPUs: Multiple warp instructions simultaneously
- **Why Have Many Warps per SM?**
    - Key purpose: ***Tolerate long-latency operations*** like global memory access
    - When a warp waits for long-latency operation:
        - That warp is not selected for execution

        - Another ready warp is selected instead
        - Priority mechanism determines selection
    - This process called ***"latency tolerance"*** or ***"latency hiding"***
- **Warp Scheduling Benefits**
    - Handles various operation latencies:
        - Pipelined floating-point arithmetic
        - Branch instructions
    - ***Zero-overhead thread scheduling***
        - No idle/wasted execution time
        - Hardware likely finds ready warps to execute
- **GPU Architecture Impact**
    - Less chip area needed for:
        - Cache memories
        - Branch prediction mechanisms
    - More chip area available for:
        - Floating-point execution
        - Memory access channels
- **Oversubscription for Effectiveness**
    - Example: Ampere A100 GPU
        - 64 cores per SM
        - Can handle up to 2048 threads simultaneously
        - *32x more threads than cores*
    - Benefits:
        - Essential for latency tolerance
        - Increases chances of finding executable warps
        - Maximizes hardware utilization
- **Threads, Context-switching, and Zero-overhead Scheduling**
    - Understanding Threads
        - Based on the **von Neumann model**, we can understand how threads are implemented
        - A **thread** in modern computers is:
            - A program and its execution state on a von Neumann Processor
            - Contains:
                - Program code
                - Current instruction being executed
                - Values of variables and data structures
    - Von Neumann Implementation
        - Code stored in memory
        - **PC (Program Counter)**: Tracks address of current instruction
        - **IR (Instruction Register)**: Holds current instruction
        - Registers and memory store variable/data values
    - Context Switching
        - Modern processors allow **context-switching**

        - Multiple threads can **time-share** a processor
        - Process:
            - Save/restore PC value
            - Save/restore register contents
            - Save/restore memory contents
        - **Key Challenge**: Significant execution time overhead from saving/restoring registers
    - Zero-overhead Scheduling

        - **GPU Feature**: Efficient warp management
            - Can put waiting warps to sleep
            - Activate ready warps
            - No extra idle cycles
        - Comparison with Traditional CPUs:
            - CPUs: Must save/load execution states to/from memory
            - GPUs: Hold all execution states in hardware registers
            - **Benefit**: No need to save/restore states between warps

---

Resource partitioning and occupancy

- **Key Concepts:**

    - **Occupancy**: **Ratio of assigned warps to maximum supported warps in an SM**
    - **Dynamic resource partitioning** across threads for efficient execution
- **SM Execution Resources:**

    - Registers
    - Shared memory
    - Thread block slots
    - Thread slots
- **Resource Allocation Example (Ampere A100 GPU):**

    - Maximum 32 blocks per SM
    - Maximum 64 warps (2048 threads) per SM
    - Maximum 1024 threads per block
- **Thread Slot Partitioning:**

    - Block size 1024 threads → 2 blocks per SM
    - Block size 512 threads → 4 blocks per SM
    - Block size 256 threads → 8 blocks per SM
    - Block size 128 threads → 16 blocks per SM
    - Block size 64 threads → 32 blocks per SM
- **Benefits of Dynamic Partitioning:**

    - Flexible execution of many small blocks or few large blocks
    - Better resource utilization compared to fixed partitioning
    - Adapts to varying thread requirements

- **Occupancy Limitations:**
  - **Block Size Impact:**
    - Block size of 32 threads → only 50% occupancy
    - Need minimum 64 threads per block for maximum occupancy
  - **Block Size Divisibility:**
    - Block size of 768 → only 75% occupancy
    - Results in 512 unutilized thread slots
  - **explain**

Occupancy Limitations

Occupancy is limited by factors such as:

- **Maximum number of threads per multiprocessor**
- **Maximum number of warps per multiprocessor**
- **Available shared memory and registers**

The occupancy percentage represents how many of the GPU's computational resources are in use relative to their maximum capacity.

Block Size Impact

Example: Block Size of 32 Threads → 50% Occupancy

- **Explanation**:
  - Each warp in CUDA consists of **32 threads**.
  - For best performance, blocks should have enough threads to fully utilize all warp slots on a multiprocessor.
  - If a block has only 32 threads, this results in just **1 warp per block**.
  - A typical multiprocessor can run **up to 2 warps concurrently per block**, leading to only 50% occupancy.
- **Real-world Example**:
  - Assume a kernel with `blockDim.x = 32` and `gridDim.x = 1`.
  - For an NVIDIA GPU with a warp limit of 64 per multiprocessor, only **32/64 = 50% of warps** are active.

Example: Minimum 64 Threads per Block for Maximum Occupancy

- **Explanation**:
  - A block size of **64 threads** results in 2 warps per block.
  - If there are enough blocks to saturate all warps on the multiprocessor, maximum occupancy is achievable.
- **Real-world Example**:
  - Assume a kernel with `blockDim.x = 64` and `gridDim.x = 10`.
  - Each block contributes 2 warps, filling up the warp slots fully on the multiprocessor.

Block Size Divisibility

Example: Block Size of 768 → 75% Occupancy

- **Explanation**:
  - GPUs have a fixed maximum number of **threads per multiprocessor** (e.g., 1024 threads).
  - If the block size (number of threads) is not divisible by this maximum, some resources remain

unutilized.

- **Real-world Example**:
  - A multiprocessor can handle 1024 threads.
  - A block with `blockDim.x = 768` threads creates **24 warps (768/32)**.
  - The GPU allocates resources for the next power-of-two warps (32 warps), leaving **8 warps unused**.
  - This results in **768/1024 = 75% occupancy**.

Example: Results in 512 Unutilized Thread Slots

- **Explanation**:
  - Using a block size that isn't a multiple of the GPU's warp size or limits results in unused thread slots.
- **Real-world Example**:
  - If the maximum number of threads per multiprocessor is 1024 and each block contains 768 threads:
    - The remaining **1024 - 768 = 256 slots** are unused in the current block.
    - For 2 such blocks scheduled, **512 slots (256 per block)** remain unutilized.

Key Insights for Optimal Occupancy

1. **Use multiples of warp size (32 threads)** for block size.
   - E.g., 128, 256, or 512 threads per block.
2. **Ensure total threads fit well into the maximum threads per multiprocessor.**
3. **Balance resources**: Consider the trade-offs between thread count, shared memory, and register usage for maximum efficiency.



- **Register Usage Considerations:**
  - Maximum 65,536 registers per SM (Ampere A100)
  - Full occupancy requires ≤32 registers per thread
  - Using >32 registers reduces maximum occupancy
  - Example:
    - 64 registers/thread → maximum 50% occupancy
    - Register spilling may increase occupancy but impact performance
- **Performance Impact Example:**
  - Scenario 1:
    - 31 registers/thread, 512 threads/block
    - Result: 100% occupancy, 4 blocks/SM
  - Scenario 2:

        - 33 registers/thread, 512 threads/block
        - Result: 75% occupancy, 3 blocks/SM
        - Demonstrates "performance cliff" effect
- **Resource Management Tools:**
    - CUDA Occupancy Calculator available for precise calculations
    - Helps determine actual thread count per SM
    - Accounts for complex resource interactions

Querying device properties

- Important Aspects to Consider:
    - Number of SMs in a device
    - Number of blocks and threads per SM
    - Other resource capabilities
- Application Design Considerations
    - Modern applications need to run on **diverse hardware systems**
    - Applications must **query available resources** to:
        - Maximize performance on capable systems
        - Adapt to less capable systems
- Compute Capability
    - Determines resources available in each SM
    - **Higher compute capability = More resources**
    - Increases with each GPU generation
    - Example: **Ampere A100 GPU has compute capability 8.0**
- CUDA Device Query System
    - **Host Code Query Mechanism**
        - Uses CUDA runtime system
        - API function: cudaGetDeviceCount

          ```
          int devCount;
          cudaGetDeviceCount(&devCount);
          ```
    - **Multiple CUDA Devices**
        - Modern PCs often have 2+ CUDA devices
        - Include "integrated" GPUs:
            - Provide basic graphics functionality
            - Limited performance for CUDA applications
            - Used for window-based interfaces
    - **Device Properties Query**

- Devices numbered from 0 to devCount-1
- Uses cudaGetDeviceProperties API
- Allows iteration through available devices

```
cudaDeviceProp devProp;
for(unsigned int i = 0; i < devCount; i++) {
   cudaError_t err = cudaGetDeviceProperties(&devProp, i);
   if (err != cudaSuccess) {
      printf("Error getting device properties: %s\n", cudaGetErrorString(err));
      return -1;
   }

   // Check device capabilities
   if (devProp.computeMode == cudaComputeModeProhibited) {
      printf("Device %d: Compute mode prohibited\n", i);
      continue;
   }
}
```

- **cudaDeviceProp** is a C struct containing CUDA device properties
- Properties accessed through **devProp** variable set by **cudaGetDeviceProperties** function

Important Fields for Thread Execution Resources:

Thread Limits

- **maxThreadsPerBlock**: Maximum threads per block
  - Up to 1024 threads on some devices
  - May vary between devices
  - Important to query for application requirements

Device Capabilities

- **multiProcessorCount**: Number of SMs in device
- **clockRate**: Device clock frequency
  - Combined with SM count indicates maximum hardware throughput

Dimensional Limits

Block Dimensions

- **maxThreadsDim[0]**: x dimension limit
- **maxThreadsDim[1]**: y dimension limit
- **maxThreadsDim[2]**: z dimension limit

Grid Dimensions

- **maxGridSize[0]**: x dimension limit
- **maxGridSize[1]**: y dimension limit
- **maxGridSize[2]**: z dimension limit
- Used to determine if grid can handle full dataset

Resource Information

- **regsPerBlock**: Available registers per SM
  - Helps determine kernel occupancy
  - Note: Name can be misleading
  - Maximum block usage may be less than SM total on some compute capabilities

- **Warp Details**
  - **warpSize**: Size of warps for the hardware